

Machine Learning – HW#2

General notes:

- It is **recommended** that you work alone to learn the material. However, groups of size 2 are allowed.
- Read and understand the questions, and answer accordingly. Be sure to answer what is asked. When in doubt, you can provide more details.
- Some of the questions require material not covered in class, or will be covered in future lectures. This is intentional.
- **Warning:** do not copy other student work. This includes code similarity. Do not sit and solve assignments together, unless it is your group partner.

Submission document format guidelines:

- Submit pdf document to Gradescope.
- You can use word, latex, or any other editor.
- No need for cover page. Provide your names and IDs in the header.
- Answer each Question on a new page. No need to copy the question, just put Question # at the beginning, then a), b), etc. for each sub-question.
- When submitting to Gradescope, you will be given the chance to map pages to questions.
- For groups, one student submits and adds their teammate during submission on Gradescope.

Question 1 [18 pts]

You are given a training set `q1_train.csv`. The dataset was generated using the following underlying data distribution:

$x_{i1} \sim \text{Uniform}(a = 0, b = 100)$ continuous uniform

$x_{i2} \sim \text{Uniform}(a = 0, b = 100)$ continuous uniform

$y_i | x_i \sim \text{Normal}(\mu = 2x_{i1} + 2x_{i2}, \sigma = 10)$

a) Code a python function `gen_sample(m)` to generate m samples (x_i, y_i) for this problem. Your function should return a pandas DataFrame with columns ['f1', 'f2', 'y']. Show code snippet.

Hint: it's easiest to generate the samples one by one in a loop. Collect the samples in a list (append sample at the end of the loop), then construct dataframe from the list.

b) Consider an ML algorithm with hypothesis space/set $\mathcal{H} = \{h_1, h_2, h_3\}$ where:

- $h_1(x_i) = x_{i1}$
- $h_2(x_i) = \sin(x_{i1}) + x_{i2}$
- $h_3(x_i) = 2x_{i1} + 2x_{i2}$

In python, use squared loss to compute the empirical loss of each hypothesis on `q1_train.csv` (show code snippet). Show the empirical loss, and then use ERM to select the hypothesis that best fits `q1_train.csv`.

Note1: for a dataset, you compute the *average* loss of samples.

Note2: you can use numpy/vectorized way to compute this without loops. Also, use `np.sin()`.

c) Using your code from (a), and same hypothesis space from (b), generate a dataset of 20,000 samples. Use ERM to select the best hypothesis that fits your generated dataset. Is it the same as in (b)? Show code snippets.

Note1: it's better not use loops to generate m samples one by one. Instead generate m feature samples in one go using the size argument. Then stack the feature vectors using `np.hstack` or any other mean to construct the dataset matrix (2D array).

d) If you were to deploy the selected hypothesis from (b) or (c), which would you choose and why?

Question 2 [16 pts]

You want to build a simple Naïve Bayes classifier for apple quality given its features. You are given the following training set:

Size	Color	Shape	Quality
medium	red	irregular	good
large	red	circle	bad
medium	green	circle	bad
large	red	irregular	good
large	red	circle	good
small	red	circle	good
large	red	circle	bad
medium	red	circle	excellent
small	green	circle	bad
small	green	circle	excellent

a) Estimate the parameters of the naïve bayes model. Use **Laplace smoothing**. Give your answer as a table for each feature (similar to what is shown below). Do not forget about the class priors.

P(Size | Quality)

Size\Quality	bad	good	Excellent
small			
medium			
large			

b) Use the trained naïve bayes model to predict on $x_{new} = [small, green, irregular]$. Show your computation to support your answer.

Question 3 [16 pts]

You are given heart-stroke training and test sets. This dataset has discrete and continuous features, and binary label.

The continuous features are: `age`, `avg_glucose_level`, and `bmi`. The rest are discrete features (categorical). The label `stroke` is binary $\{0, 1\}$.

a) Use scikit-learn to train Naïve Bayes model on `q3_train.csv`. Note that you will need to use two models: one for the discrete features (CategoricalNB) and one for the continuous features (GuassianNB). Use default parameters (no changes needed). Show the code snippet for reading in training set, setting up the feature matrices and label vectors, initializing the two scikit-learn algorithms, and fitting to the train data.

Note: For CategoricalNB, you will need to see how to convert discrete string features into numbers. You can use [OrdinalEncoder](#).

b) To predict a label:

- You will need to use [predict_proba](#) method on both models to compute $p(y \mid x_{discretefeatures})$ and $p(y \mid x_{continuousfeatures})$.
- Multiply these two probabilities to get a result that is proportional to $p(y \mid x)$.
- Take the argmax class to predict the label. You can use `np.argmax` from numpy. Also, note you can know the order of classes used by sklearn model using `model.classes_` (see `classes_` description [here](#)).

Predict on the training set and compute the accuracy using scikit-learn (see [link](#)). Show code snippet for predicting labels on the training set and accuracy compute.

c) Using the trained model, predict labels for the test set `q3_test.csv` and compute the accuracy (show code snippet). Comment on the train accuracy vs test accuracy.

Note: use the *fitted* OrdinalEncoder from part A that is fit on the train data to *transform* the test features.